

Back To Practice

< Q1 >

Save

Banker's Deadlock Avoidance Algorithm

Completed

Write a C program to Implement the Banker's Deadlock Avoidance Algorithm. The program should accept the number of processes and resource types, the Maximum Demand matrix, the currently Allocated resources matrix, and the Available resources vector. Calculate the 'Need' matrix and determine if the system is in a 'Safe State'. If it is safe, display the Safe Sequence; otherwise, indicate that a deadlock may occur.

Input Format

1. The first line of input contains an integer representing the number of processes.
2. The second line contains an integer representing the number of resource types.
3. The next lines contain the Allocation matrix, where each row corresponds to a process and each column corresponds to a resource type.
4. This is followed by the Maximum Demand (Max) matrix with the same dimensions.
5. The final line contains the Available resources vector, where each value represents the number of currently available instances of each resource type.

Select Language : C (GCC 9.2.0)

```
1 #include <stdio.h>
2
3 int main() {
4     int n, m;
5
6     printf("Enter number of processes: \n");
7     scanf("%d", &n);
8     printf("Enter number of resource types: \n");
9     scanf("%d", &m);
10
11     int alloc[n][m], max[n][m], need[n][m], avail[m];
12
13     printf("Enter Allocation Matrix:\n");
14     for (int i = 0; i < n; i++)
15         for (int j = 0; j < m; j++)
16             scanf("%d", &alloc[i][j]);
17
18     printf("Enter Max Matrix:\n");
19     for (int i = 0; i < n; i++)
20         for (int j = 0; j < m; j++)
21             scanf("%d", &max[i][j]);
22
23     printf("Enter Available resources vector:\n");
24     for (int j = 0; j < m; j++)
25         scanf("%d", &avail[j]);
26 }
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

Custom Test

Success

Input :

Expected Output :

Back To Practice

Q1

Save

Banker's Deadlock Avoidance Algorithm

Write a C program to implement the Banker's Deadlock Avoidance Algorithm. The program should accept the number of processes and resource types, the Maximum Demand matrix, the currently Allocated resources matrix, and the Available resources vector. Calculate the 'Need' matrix and determine if the system is in a 'Safe State'. If it is safe, display the Safe Sequence; otherwise, indicate that a deadlock may occur.

Input Format

- The first line of Input contains an integer representing the number of processes.
- The second line contains an integer representing the number of resource types.
- The next lines contain the Allocation matrix, where each row corresponds to a process and each column corresponds to a resource type.
- This is followed by the Maximum Demand (Max) matrix with the same dimensions.
- The final line contains the Available resources vector, where each value represents the number of currently available instances of each resource type.

Select Language: C (GCC 9.2.0)

```
18 printf("Enter Max Matrix:\n");
19 for (int i = 0; i < n; i++)
20     for (int j = 0; j < m; j++) {
21         scanf("%d", &max[i][j]);
22         need[i][j] = max[i][j] - alloc[i][j];
23     }
24
25 printf("Enter Available Resources: \n");
26 for (int i = 0; i < m; i++)
27     scanf("%d", &avail[i]);
28
29 int finish[n], safeSeq[n];
30 for (int i = 0; i < n; i++) finish[i] = 0;
31
32 int work[m];
33 for (int i = 0; i < m; i++) work[i] = avail[i];
34
35 int count = 0;
36 while (count < n) {
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

Custom Test

Input:

Expected Output:

Success

Back To Practice

Q1

Save

Banker's Deadlock Avoidance Algorithm

Write a C program to implement the Banker's Deadlock Avoidance Algorithm. The program should accept the number of processes and resource types, the Maximum Demand matrix, the currently Allocated resources matrix, and the Available resources vector. Calculate the 'Need' matrix and determine if the system is in a 'Safe State'. If it is safe, display the Safe Sequence; otherwise, indicate that a deadlock may occur.

Input Format

1. The first line of input contains an integer representing the number of processes.
2. The second line contains an integer representing the number of resource types.
3. The next lines contain the Allocation matrix, where each row corresponds to a process and each column corresponds to a resource type.
4. This is followed by the Maximum Demand (Max) matrix with the same dimensions.
5. The final line contains the Available resources vector, where each value represents the number of currently available instances of each resource type.

Select Language : C (GCC 9.2.0)

```

47         found = 1;
48     }
49 }
50 }
51 if (!found) break;
52 }
53
54 if (count == n) {
55     printf("System is in a SAFE STATE.\n");
56     printf("Safe Sequence:");
57     for (int i = 0; i < n; i++) printf(" %d", safeseq[i]);
58     printf("\n");
59 } else {
60     printf("System is NOT in a safe state (Deadlock may occur).\n");
61 }
62
63 return 0;
64 }
    
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

Custom Test

Input :

Expected Output :

Success